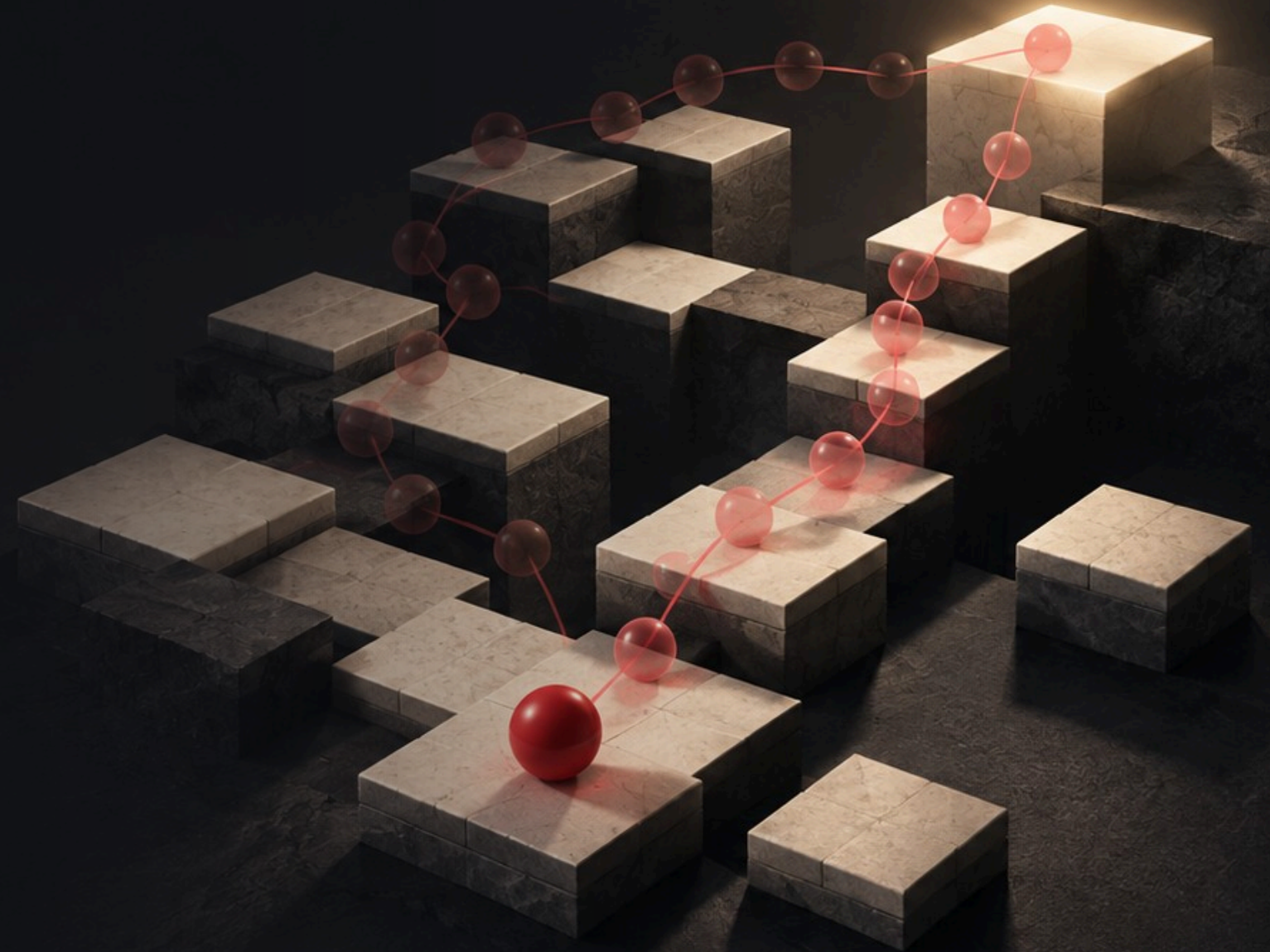


Reinforcement learning

Unit 6 · Intelligent Systems

School of Computer Science · Ciudad Real
Universidad de Castilla-La Mancha

Rubén Juárez Cádiz



Learning connects decisions with their consequences

60 slides · Five interactive laboratories · Worked exercises

01

Formulate

State, action, reward and objective.

02

→ Evaluate

Return, values and Bellman backups.

03

→ Learn

Monte Carlo, TD and action values.

04

→ Test

Exploration, evaluation and failure modes.

By the end, explain an update numerically and justify what an experiment does and does not show.

Search plans; reinforcement learning improves behaviour

Course idea	Main question	What must be available?
Search	Which path reaches the goal?	A state model and legal transitions.
Adversarial search	Which move survives a reply?	Game rules and an opponent assumption.
Reinforcement learning	Which behaviour earns return?	Experience and a reward definition.

Planning and learning can coexist. A learned model can support search.

The learning signal changes the problem

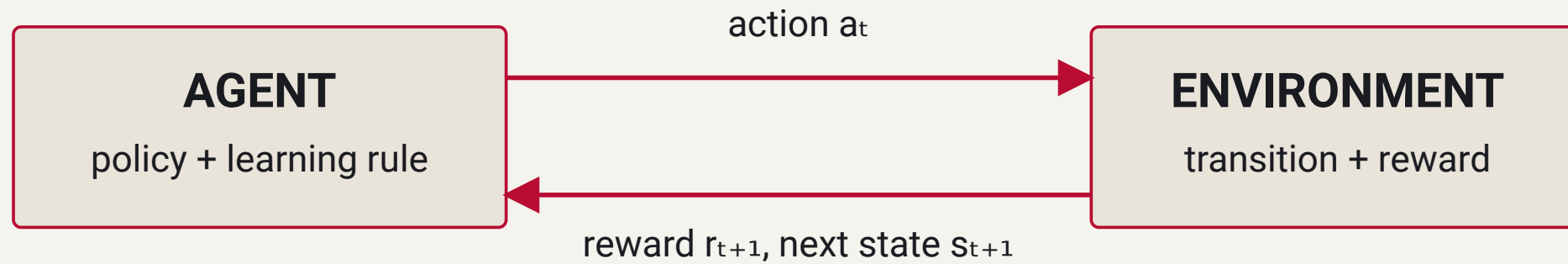
Supervised learning Examples provide target outputs, such as image labels.

Unsupervised learning Data reveal structure without a supplied target label.

Reinforcement learning Actions influence later data, and rewards evaluate their consequences.

A reward usually gives no explicit label for the best action in the current state.

The reward arrives after the action



At time t : observe s_t , choose a_t , then receive r_{t+1} and s_{t+1} .

An MDP defines a sequential decision problem

Element	Definition in this unit
S, A(s)	States and legal actions in each state.
P(s' s,a)	Probability of the next state after an action.
R(s,a,s')	Expected immediate reward for that transition.
γ and start distribution	Discount factor and where episodes begin.
Terminal states	States after which no further task rewards occur.

The policy π is what we evaluate or improve; it is separate from the environment model.

A useful state retains what predicts the next step

$$P(s',r \mid \text{history},a) = P(s',r \mid s,a)$$

Enough information

A robot state may need position, velocity and remaining battery.

Missing information

A single camera frame may hide motion or an obstacle behind the robot.

For a task with a true deadline, include remaining time when it changes the decision problem.

One gridworld, fully specified

(1,1) •	(1,2) •	(1,3) •	(1,4) +1
(2,1) •	Wall	(2,3) •	(2,4) -1
(3,1) Start	(3,2) •	(3,3) •	(3,4) •

Four actions

Up, right, down and left.

Movement

Default: 0.8 intended, 0.1 to each side. A wall or boundary keeps the agent in place.

Reward on entry

+1 or -1 for a terminal. Otherwise -0.04, including a blocked move.

Start: (3,1). $\gamma=0.9$. Terminal values are 0: their reward has already been received on entry.

A policy specifies the agent's choices

Deterministic

$$\mathbf{a} = \boldsymbol{\pi}(\mathbf{s})$$

Each state selects one action.

Stochastic

$$\boldsymbol{\pi}(\mathbf{a} \mid \mathbf{s})$$

Each state defines a probability distribution over legal actions.

For every state, probabilities must be nonnegative and sum to 1.

Return accumulates rewards over time

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

Reward	The immediate signal from one transition.
---------------	---

Return	The discounted total along a trajectory.
---------------	--

Objective	Maximize expected return under the chosen start distribution.
------------------	---

One lucky episode does not establish a high expected return.

A delayed payoff can justify an initial cost

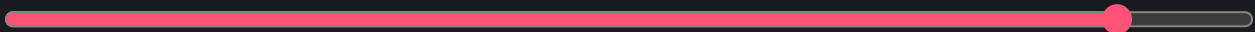
Option	Reward sequence	Return at $\gamma=0.9$
Quick	[1] then terminate	1
Patient	[-1, -1, 5] then terminate	$-1 - 0.9 + 0.9^2 \times 5 = 2.15$

$$G_t = R_{t+1} + \gamma G_{t+1}$$

The last return is 5. Working backwards gives 3.5, then 2.15.

Discounting can change the preferred trajectory

Discount γ : 0.900000



Quick 1.000



Patient 2.150



$\gamma=0$

$\gamma=0.9$

Exact tie

Patient: weighted rewards

-1	$-\gamma$	$5\gamma^2$
-1.000	-0.900	4.050

Patient has the larger return. Patient – Quick = 1.150.

Quick: [1]. Patient: [-1, -1, 5]. Both end after the listed rewards.

Discounting and termination solve different problems

Continuing task

For bounded rewards and $\gamma < 1$, the discounted infinite sum is finite.

$$|G_t| \leq R_{\max} / (1 - \gamma)$$

Episodic task

Return stops at a task terminal. $\gamma = 1$ can be valid when returns are well defined.

A small γ changes preferences over timing. It does not make the task end sooner.

A reward defines the preference we ask the agent to learn

Sparse reward

Only goal or failure outcomes provide feedback.

Step cost

Charges time or effort and can favour shorter successful routes.

Proxy reward

Measures a convenient substitute that may reward unwanted behaviour.

The algorithm attempts to optimize the specified return. Success on that signal can still miss the intended goal.

Potential-based shaping can preserve optimal choices

$$R' = R + \gamma \Phi(s') - \Phi(s)$$

Idea

Add progress information through a state potential Φ .

Boundary convention

Use the same γ . For a stopped episodic return, set terminal potentials to 0.

The added discounted rewards telescope to $-\Phi(s_0)$, independent of the actions taken.

V evaluates a state; Q evaluates an action in a state

$$V^{\pi}(s) = E_{\pi}[G_t \mid S_t=s]$$

$$Q^{\pi}(s,a) = E_{\pi}[G_t \mid S_t=s, A_t=a]$$

Q fixes the first action, then follows π . V averages over the policy's first action.

A fixed policy averages over actions and outcomes

$$V^{\pi}(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s,a) [R(s,a,s') + \gamma V^{\pi}(s')]$$

Prediction asks how good the specified policy is.

Control selects an action before averaging its outcomes

$$V^*(s) = \max_a \sum_{s'} P(s'|s,a) [R(s,a,s') + \gamma V^*(s')]$$

The agent chooses the action. The environment samples its consequence.

One noisy action combines three possible outcomes

Outcome of RIGHT from (1,3)	Probability	Reward + $\gamma V(\text{next})$
Enter +1 terminal	0.8	$1 + 0.9 \times 0 = 1$
Hit top boundary; stay at (1,3)	0.1	$-0.04 + 0.9 \times 0.8 = 0.68$
Move down to (2,3)	0.1	$-0.04 + 0.9 \times 0.4 = 0.32$

$Q_{\text{backup}} = 0.8 \times 1 + 0.1 \times 0.68 + 0.1 \times 0.32 = 0.90$

Value iteration repeatedly applies optimality backups

```
V[s] = 0 for every state
repeat:
    old = copy(V)
    for each nonterminal state s:
        V[s] = max_a Σ_s' P(s'|s,a) * (R + γ*old[s'])
    Δ = max_s |V[s] - old[s]|
```

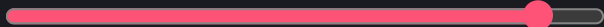
Synchronous sweeps use the previous vector for every state in the next vector.

Watch values spread through the grid

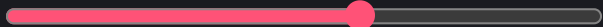
(1,1) 0.610 →	(1,2) 0.766 →	(1,3) 0.928 →	(1,4) +1 terminal · V=0
(2,1) 0.487 ↑	Wall	(2,3) 0.585 ↑	(2,4) -1 terminal · V=0
(3,1) Start 0.374 ↑	(3,2) 0.327 →	(3,3) 0.428 ↑	(3,4) 0.189 ←

Cells show V_k . Arrows are greedy with respect to V_k . Terminal labels show entry rewards.

Discount γ
0.90



Intended move p
0.8



One sweep

10 sweeps

Converge

Reset

Sweeps

25

Last sweep change Δ

1.58e-8

Start value $V(3,1)$

0.37385

Residual $\|TV-V\|_\infty$

6.91e-9

Residual below 10^{-8} . Value error is bounded by $\text{residual}/(1-\gamma) = 6.91\text{e-}8$.

Evaluation and improvement can be separated

01

Evaluate π

Solve or iterate the Bellman expectation equations.

02

→ Improve π

Choose actions greedy with respect to the evaluated values.

03

→ Repeat

Stop when the policy no longer changes.

Policy evaluation follows the current policy. Policy improvement asks whether another action is better.

A model enables averages; a sample supplies one outcome

Method family	What the update consumes	Main limitation
Known-model planning	Transition probabilities and rewards.	Requires a usable model.
Learned-model RL	Experience to fit a model, then planning.	Model errors influence the plan.
Model-free RL	Observed transitions to update values or a policy.	May require many interactions.

A simulator can generate samples without giving the learner its probability table.

A complete episode provides a sampled return

```
follow a fixed policy  $\pi$  until termination
compute G backward through the episode
for each state on its first visit in the episode:
    N[s] += 1
    V[s] += (G_at_that_visit - V[s]) / N[s]
```

First-visit Monte Carlo averages one return per visited state per episode.

The return becomes a target after the episode ends

Observation	Value
Episode rewards	-1, -1, 5
Discount and sampled return	$\gamma=0.9$, $G_0=2.15$
Old estimate and learning rate	$V(s)=0.5$, $\alpha=0.2$
Update	$0.5 + 0.2(2.15-0.5) = 0.83$

This constant-step update illustrates one sample. A running mean instead uses $\alpha=1/N(s)$.

TD(0) learns before the episode finishes

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$

$$V(S_t) \leftarrow V(S_t) + \alpha \delta_t$$

Sample

The observed reward and successor state.

Bootstrap

The current estimate of that successor's value.

For a true terminal successor, the continuation value is zero.

One transition corrects one estimate

Quantity	Calculation
Known sample	$r = -0.04, V(s) = 0.5, V(s') = 2$
Target at $\gamma = 0.9$	$-0.04 + 0.9 \times 2 = 1.76$
TD error	$\delta = 1.76 - 0.5 = 1.26$
New value at $\alpha = 0.2$	$0.5 + 0.2 \times 1.26 = 0.752$

The sample updates $V(s)$. It does not directly update $V(s')$.

Monte Carlo and TD use different targets

Aspect	Monte Carlo prediction	TD(0) prediction
Target	Complete sampled return G_t .	$r + \gamma V(s')$.
When available	After the episode completes.	After one transition.
Bootstraps?	No.	Yes.
Typical trade-off	Return can have high variance.	Target depends on imperfect estimates.

Neither method is universally better. Compare them on the same policy and task.

Q-learning uses a greedy continuation target

$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

Behaviour

The agent can explore while collecting transitions.

Target

The update uses the largest current next-state action value.

Q-learning is off-policy: the target can differ from the action actually selected next.

SARSA includes the selected next action

$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma Q(s',a') - Q(s,a)]$$

Five quantities

State, action, reward, next state, next action.

On-policy

The next action comes from the behaviour policy being learned.

If exploration selects a poor next action, that action affects this SARSA target.

Compare three one-step update targets

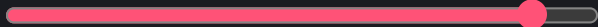
Transition

Ordinary: reward -0.04

Learning rate α
0.20



Discount γ
0.90



Selected next action for SARSA

Right: Q=1

Current estimate=0.5. $V(s')=2$.
 $Q(s',\cdot)=[4,1,-2,0]$. These are illustrative estimates.

Method	Target	New estimate
TD(0)	1.760	0.752
Q-learning	3.560	1.112
SARSA	0.860	0.572

SARSA uses Right. Q-learning uses the maximum next value 4. TD uses $V(s')=2$.

Reset example

ϵ -greedy reserves probability for every action

$$\pi(a|s) = (1-\epsilon) \cdot \mathbb{1}[a=a^*] + \epsilon/|A(s)|$$

Four actions, one chosen greedy action

Probability at $\epsilon=0.2$

Chosen greedy action

$0.8 + 0.2/4 = 0.85$

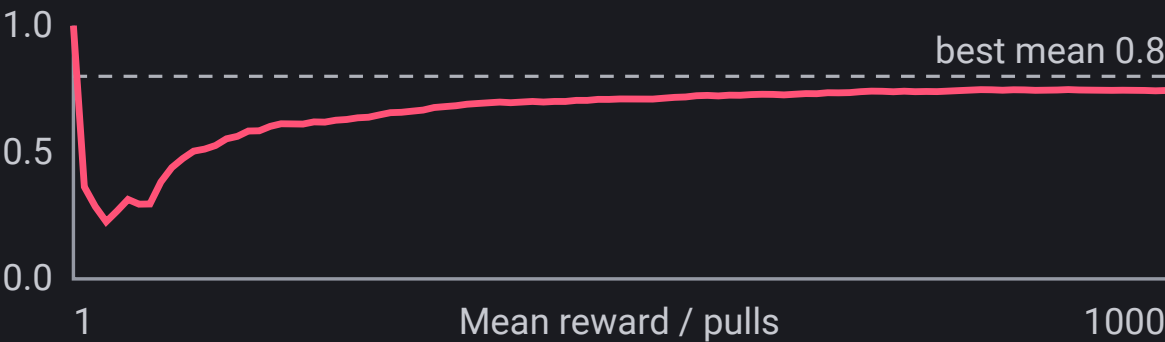
Each other action

$0.2/4 = 0.05$

The random branch can select the greedy action too.

Explore three actions with unknown reward rates

Action	True p	$\hat{Q}$	Pulls
A	0.2	0.313	99
B	0.5	0.561	41
C	0.8	0.802	860



Reward is 1 with probability p , otherwise 0. True p is visible to you; the agent uses only $\hat{Q}$ and ϵ .

Exploration ϵ

0.1

Seed

1

1 pull

100 pulls

1,000 pulls

Reset

Pulls

1,000

Mean reward

0.744

Best-action share

86.0%

Expected regret

71.7

Last: action C, reward 1. Seed 1. Change ϵ or seed to start a new run.

Bandits isolate exploration from delayed state effects

Bandit laboratory

Every pull returns to the same decision setting.

The action changes the current reward distribution.

Sequential MDP

The action also changes which states come next.

Exploration may open or close future opportunities.

A strategy that works in a bandit can still struggle to reach informative states in an MDP.

A Q-learning loop needs two stopping decisions

```
s = env.reset()
while collecting data:
    a = epsilon_greedy(Q[s])
    s_next, r, terminated, truncated = env.step(a)
    target = r if terminated else r +  $\gamma$ *max(Q[s_next])
    Q[s,a] +=  $\alpha$  * (target - Q[s,a])
    s = env.reset() if (terminated or truncated) else s_next
```

Reset after either ending signal. Remove continuation only for true termination.

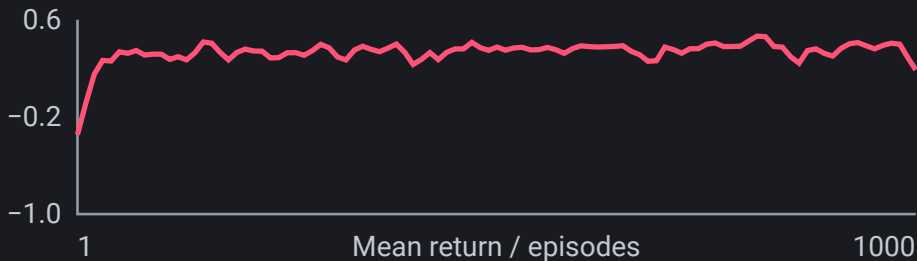
A data-collection cutoff is not a task terminal

Reason for stopping	Bootstrap?	Example target
Goal reached or task failure	No.	r
External collection limit	Yes, if the task would continue.	$r + \gamma V(s')$
Deadline inside the task	No after its true final step.	r ; include remaining time in state.

With $r=1$, $\gamma=0.9$ and next value 5: termination gives 1; external truncation gives 5.5.

Train a policy from sampled gridworld experience

(1,1) 0.734 →	(1,2) 0.860 →	(1,3) 1.000 →	(1,4) +1 terminal · V=0
(2,1) 0.621 ↑	Wall	(2,3) 0.860 ↑	(2,4) -1 terminal · V=0
(3,1) Start 0.519 ↑	(3,2) 0.621 →	(3,3) 0.734 ↑	(3,4) 0.370 ←



Cells: $\max_a Q(s,a)$, with greedy arrows. Chart: training return, averaged over up to 20 recent episodes.

Algorithm

Q-learning

Motion

Deterministic

ϵ

0.3

α

0.3

Seed

17

1 episode

100 episodes

1,000 episodes

Evaluate 100

Reset

Training episodes

1,000

Transitions

7,170

Latest: goal, 5 steps, return 0.519. $\gamma=0.9$. Rollouts stop after at most 100 steps.

Frozen evaluation: 100/100 goals, 0 failures, 0 caps. Mean return 0.51854. $\epsilon=0$; no updates.

Training return and evaluation return answer different questions

Training curve	Performance while exploring and changing the estimates.
-----------------------	---

Frozen evaluation	Performance of the current greedy policy without learning.
--------------------------	--

Learned cell value	max Q is an estimate, not an exact success probability.
---------------------------	---

Model baseline	At $p=1$ and $\gamma=0.9$, optimal start return is 0.51854.
-----------------------	--

A deterministic optimal route has five actions: four rewards of -0.04 , then $+1$.

Tabular Q-learning guarantees need conditions

Stable finite MDP	Finite state-action table, stationary dynamics and bounded rewards.
--------------------------	---

Discounted setting	Use $\gamma < 1$ for the standard discounted theorem.
---------------------------	---

Coverage	Visit every relevant state-action pair infinitely often.
-----------------	--

Learning rates	For each pair: $\sum \alpha_n = \infty$ and $\sum \alpha_n^2 < \infty$.
-----------------------	--

A finite classroom run with constant α illustrates learning; it does not satisfy an asymptotic theorem.

α changes estimation; γ changes the objective

Parameter	Effect	Common mistake
α , learning rate	Weights a new target against the old estimate.	Reducing α so quickly that learning effectively stops.
γ , discount	Weights rewards at different times.	Treating γ as a measure of confidence in the data.
ϵ , exploration	Changes the behaviour that collects experience.	Assuming a greedy policy has already learned every action.

A parameter sweep needs equal interaction budgets and multiple random seeds.

On-policy and off-policy describe the learning target

Method	Continuation in the target	Relationship to behaviour
SARSA	The sampled next action a' .	On-policy when that action comes from the learned behaviour.
Q-learning	$\max_a Q(s',a')$.	Targets greedy continuation while behaviour may explore.
Expected SARSA	$\sum_a \pi(a s')Q(s',a')$.	Averages the specified target policy's next action.

Off-policy does not mean “offline”. Offline learning uses a fixed previously collected dataset.

Exploration changes the cost of a risky route

Route near a cliff

Short when actions are greedy, but exploratory moves may incur a large penalty.

Route with a buffer

Longer, but exploratory actions have less severe consequences.

SARSA can account for its ongoing exploration. Q-learning targets greedy continuation.

A reliable result needs repeated, comparable experiments

Freeze the policy	Evaluate without parameter updates and state the action-selection rule.
Repeat training	Use independent seeds and report uncertainty across runs.
Keep budgets equal	Compare interactions, computation and tuning effort.
Test the intended task	Report return, success, failures and performance under changed conditions.

One best seed or one smooth curve is an incomplete comparison.

A proxy can reward behaviour that never completes the task

Intended objective

Deliver a parcel successfully.

Flawed proxy

Give +1 whenever the robot enters the loading bay.

A repeated exit-and-return loop may collect reward indefinitely.

Test the reward on deliberate shortcuts before attributing the failure to the learning algorithm.

The state and action space constrain what can be learned

Design choice	Consequence
Battery omitted from state	Identical positions can require different decisions.
Actions ignore legal constraints	The learner may select impossible moves.
Goal bonus repeats	A loop can outperform finishing.
Remaining deadline omitted	A finite-horizon task can appear non-Markovian.

An incorrect task formulation can make a correct update learn the wrong behaviour.

Loopover supports both planning and learning formulations

Search formulation

Board configuration, legal row/column shifts, known transitions and a goal test.

RL formulation

The same board and moves, with a step cost, terminal success reward and training distribution.

For a single small known puzzle, direct search is a strong baseline. Training may help across many instances.

Large state spaces require shared representations

Table

Function approximator

$Q[s,a]$

$Q(s,a;\theta)$ or $\pi(a|s;\theta)$

Separate estimates for discrete state-action pairs.

Parameters generalize across inputs.

Generalization reduces storage, but an update can also change predictions for other states.

DQN combines approximation with two stabilizing devices

Replay buffer	Store transitions and sample training batches from past experience.
----------------------	---

Target network	Use a separate, more slowly updated network to build the target.
-----------------------	--

Learning target	$r + \gamma \max_a Q(s', a; \theta^-)$, with zero continuation at termination.
------------------------	---

These mechanisms improve the training procedure; they do not make deep RL universally stable.

A policy can be optimized directly

$$\pi(a | s; \theta)$$

Policy parameters

A network produces action probabilities or a continuous-action distribution.

Return signal

Sampled returns help adjust how likely different actions become.

The objective remains expected return, even when the algorithm no longer stores a Q-table.

The critic supplies a learning signal for the actor

Actor

The parameterized policy chooses actions.

Critic

A value estimator helps judge whether the observed outcome was better than expected.

$$A^{\pi}(s,a) = Q^{\pi}(s,a) - V^{\pi}(s)$$

Advantage compares an action with the policy's usual outcome in that state.

Learning performance is only one requirement

Interaction cost	Training may require far more experience than deployment can afford.
-------------------------	--

Unsafe exploration	Some actions are unacceptable even during learning.
---------------------------	---

Distribution shift	New states or changed dynamics can expose brittle behaviour.
---------------------------	--

Validation	A high average return may conceal rare, severe failures.
-------------------	--

	The task determines whether simulation, constrained actions or another method is appropriate.
--	---

Formulate a delivery robot as an MDP

Task	Deliver one parcel before a 20-step deadline.
-------------	---

Available facts	Grid location, battery level, parcel status and remaining time.
------------------------	---

Your design	States, legal actions, terminal conditions, transitions and rewards.
--------------------	--

Your evaluation	State one success metric and one failure metric.
------------------------	--

	Explain what changes if movement is noisy or the battery level is hidden.
--	--

A complete state makes the deadline explicit

Element	Example answer
State	(cell, battery, parcel status, steps remaining).
Actions	Legal moves, pick up, deliver and recharge where allowed.
Dynamics	Movement noise and battery consumption are specified.
End and reward	+10 for delivery, -10 for failure, otherwise -0.1; $\gamma=1$.
Evaluation	Delivery rate and battery/deadline failures on held-out maps.

Rewards here are alternatives on a transition. The finite 20-step horizon makes the return bounded.

Calculate the targets before choosing an algorithm

One nonterminal sample

$Q(s,a)=2, r=-1, \gamma=0.9, \alpha=0.5.$

$Q(s',\cdot)=[4,1].$

The selected next action has value 1.

Questions

1. Compute the Q-learning update.
2. Compute the SARSA update.
3. Repeat both if s' is truly terminal.

For an ϵ -greedy extension, use $\epsilon=0.2$ and two legal actions.

The continuation determines the difference

Method	Target	Error	New Q
Q-learning	$-1 + 0.9 \times 4 = 2.6$	0.6	2.3
SARSA	$-1 + 0.9 \times 1 = -0.1$	-2.1	0.95
Either, true terminal	-1	-3	0.5
Expected SARSA, $\epsilon=0.2$	$-1 + 0.9 \times 3.7 = 2.33$	0.33	2.165

Expected SARSA uses probabilities 0.9 and 0.1: $0.9 \times 4 + 0.1 \times 1 = 3.7$.

Two plausible lines can create a silent learning error

```
next_state, reward, terminated, truncated = env.step(action)
done = terminated or truncated
if done:
    next_state = env.reset()
target = reward +  $\gamma$  * (1 - done) * max(Q[next_state])
Q[state, action] +=  $\alpha$  * (target - Q[state, action])
```

Identify the meaning of the bootstrap mask and which observation belongs in the target.

The update uses the final transition before resetting

```
final_state, reward, terminated, truncated = env.step(action)
if terminated:
    target = reward
else:
    target = reward +  $\gamma$  * max(Q[final_state])
Q[state, action] +=  $\alpha$  * (target - Q[state, action])
state = env.reset() if (terminated or truncated) else final_state
```

Also check legal actions, reward timing, random seeds and evaluation without learning.

Six questions to test the core ideas

01

Reward or return?

Reward belongs to one transition. Return sums discounted rewards.

02

What does γ change?

The relative weight of later rewards, hence the objective.

03

What bootstraps?

TD, SARSA and Q-learning use estimated continuation values.

04

Why does $\epsilon=0$ risk failure?

An action with a poor early estimate may never be tried again.

05

Q-learning versus SARSA?

Greedy next value versus the value of the selected next action.

06

What happens at truncation?

Keep valid continuation and use the final observation before reset.

Core reading and sources for the extensions

Course material

Supplied UCLM Unit 6 lecture, 32 slides. Expanded examples and executable labs are original to this version.

Sutton & Barto (2018)

Reinforcement Learning: An Introduction, 2nd ed. Chapters 2–6 and 13.

incompleteideas.net/book/the-book-2nd.html

Watkins & Dayan (1992)

Q-learning. *Machine Learning* 8, 279–292.

doi.org/10.1007/BF00992698

Ng, Harada & Russell (1999)

Policy invariance under reward transformations.

Berkeley: original shaping paper

Berkeley CS188

MDPs, value iteration and reinforcement learning.

inst.eecs.berkeley.edu/~cs188/textbook/

Farama Foundation

Gymnasium: Handling time limits.

gymnasium.farama.org · Handling time limits

Mnih et al. (2015)

Human-level control through deep reinforcement learning.

doi.org/10.1038/nature14236

Agarwal et al. (2021)

Deep RL at the edge of the statistical precipice.

agarwl.github.io/rliable/

An intelligent agent needs a model of what matters

Represent

Choose state, actions, goals, constraints and uncertainty.

Decide

Search, plan against opponents or evaluate a policy.

Learn

Use experience to improve predictions and choices.

Validate

Check assumptions, outcomes and failure cases.

The six units now form one toolkit: formulate the problem, choose a method and test the evidence.